

Building a RAG Document Assistant with Postgres and pgvector

After building a local LLM serving layer, the next step is giving the model access to trusted context.

A model-serving API can generate text, stream responses, and expose inference behind a clean interface. But on its own, the model only knows what is already inside its training data or what the user provides in the prompt. For many real-world AI systems, that is not enough.

Enterprise AI applications need access to private documents, technical notes, project documentation, policies, runbooks, specifications, data contracts, and internal knowledge bases.

That is where retrieval-augmented generation, or RAG, becomes essential.

The `04-rag-document-assistant` project builds a local RAG system using FastAPI, Postgres, `pgvector`, document ingestion, chunking, embeddings, vector search, and a source-grounded `/ask` endpoint.

The goal is straightforward: turn local Markdown, text, and PDF documents into searchable, retrievable context that can support grounded answers.

From an AI Data Platform Architect's perspective, this project matters because RAG is not just a model interaction pattern. It is a data architecture pattern. The quality of the final answer depends on how well the platform manages documents, chunks, embeddings, metadata, retrieval, prompt construction, and evaluation.

Why RAG Matters

Most AI applications eventually run into the same limitation: the model does not automatically know the information that matters most to the business.

It does not automatically know your internal documentation.

It does not automatically know your data model definitions.

It does not automatically know your architecture decisions.

It does not automatically know your runbooks, operating procedures, or project notes.

It does not automatically know the latest version of a technical spec or policy document.

RAG solves this by connecting language models to external knowledge.

Instead of asking the model to answer from memory, the platform retrieves relevant source material, injects that context into a prompt, and asks the model to answer from the retrieved evidence.

That changes the architecture.

The model is no longer the only intelligence layer. The retrieval system becomes just as important.

A strong RAG platform needs to answer several questions:

How are documents loaded?

How is text normalized?

How are documents split into chunks?

How are embeddings generated?

Where are embeddings stored?

How is similarity search performed?

How are retrieved chunks filtered and ranked?

How is source context inserted into the prompt?

How does the system know when it does not have enough evidence?

How is retrieval quality evaluated over time?

The RAG Document Assistant is a local implementation of those questions.

System Overview

At a high level, the project has two major flows.

The first flow is document ingestion.

A document is loaded, normalized, split into overlapping chunks, embedded, and stored in Postgres with `pgvector`.

The second flow is question answering.

A user question is embedded, searched against stored chunk vectors, converted into source-grounded prompt context, and optionally sent to the local LLM API for answer generation.

The core flow looks like this:

```
Document
-> load Markdown/text/PDF
-> normalize content
-> split into overlapping chunks
-> generate embeddings
-> store documents and chunks in Postgres + pgvector

User question
-> embed query
-> search pgvector
-> retrieve relevant chunks
-> build source-grounded prompt
-> optionally call local LLM API
-> return answer with sources
```

The major components are:

Component	Role
FastAPI API	Provides ingestion, search, document listing, health, and RAG endpoints
Postgres	Stores document metadata, chunk text, source paths, timestamps, and hashes
pgvector	Stores and searches embedding vectors using cosine distance
Document loader	Reads Markdown, text, and PDF documents
Chunker	Splits text into overlapping chunks
Embedding provider	Creates vectors using hash-based or Ollama embeddings
Vector search layer	Searches chunks and returns similarity-ranked context
Prompt builder	Formats retrieved chunks into a grounded RAG prompt
Optional LLM API	Connects to the local LLM wrapper through <code>LLM_API_URL</code>

This architecture keeps retrieval infrastructure separate from generation infrastructure. That separation is important because retrieval should be testable on its own. If the final answer is poor, the first question should be whether the right context was retrieved.

Document Ingestion

The first major capability is document ingestion.

The project supports Markdown, plain text, and PDF files. When a document is ingested, the system loads the file, extracts text, normalizes whitespace, calculates a SHA-256 content hash, and creates a durable document record.

The ingestion endpoint is:

```
POST /documents/ingest
```

A request includes:

- file path
- optional title
- optional metadata

The response includes:

- document ID
- source path
- title
- number of chunks created
- content hash

This is the point where raw files become platform-managed records.

That transition matters. In a real AI data platform, documents cannot be treated as loose files. They need identity, metadata, lineage, freshness indicators, deduplication logic, and retrievable source references.

The content hash is especially important because it creates a foundation for deduplication. Without deduplication, the same document can be indexed multiple times. That can pollute search results, cause repetitive retrieval, and create confusing citations.

From an architecture perspective, ingestion is not just a file-loading step. It is the start of document governance.

Chunking Strategy

After a document is loaded and normalized, the system splits it into chunks.

Chunking is one of the most important design choices in a RAG system.

If chunks are too small, retrieval may return fragments that lack enough surrounding context.

If chunks are too large, each vector may represent too many concepts, making similarity search less precise.

This project uses overlapping text chunks with two primary settings:

Setting	Default	Purpose
<code>CHUNK_SIZE</code>	900 characters	Keeps retrieved context focused
<code>CHUNK_OVERLAP</code>	150 characters	Reduces the chance of splitting useful context at boundaries

Overlap is a practical compromise. It helps preserve continuity when important information spans two adjacent chunks.

But overlap also increases duplicate content. Too much overlap can cause repetitive retrieval results, where several top-ranked chunks contain nearly the same text. Too little overlap can cause the system to lose important context at chunk boundaries.

That is why chunking is retrieval design.

The way documents are split directly affects what the model can later answer.

For an AI Data Platform Architect, chunking should be treated as a tunable system parameter, not a one-time preprocessing choice. Different content types may need different chunking strategies. Technical documentation, policy documents, meeting notes, source code, and data dictionaries may not all perform best with the same chunk size or overlap.

Embeddings

Once chunks are created, the system generates embeddings.

Embeddings convert text into vectors that can be compared mathematically. The purpose is to represent meaning in a way that supports semantic search.

A user may not ask a question using the exact same words that appear in the document. Vector search helps retrieve relevant passages based on semantic similarity rather than exact keyword matching.

The project supports two embedding providers:

Provider	Purpose
<code>hash</code>	Deterministic local smoke-test embeddings with no model dependency
<code>ollama</code>	Optional semantic embeddings using an Ollama embedding model

The default hash provider is intentionally simple. It allows the ingestion flow, API routes, database schema, and vector search mechanics to be tested locally without requiring a real embedding model.

That makes the project easier to run and easier to debug.

For stronger semantic retrieval, the project can use Ollama embeddings:

```
ollama pull nomic-embed-text
EMBEDDING_PROVIDER=ollama
OLLAMA_EMBEDDING_MODEL=nomic-embed-text
EMBEDDING_DIMENSION=768
```

This introduces an important production consideration: embedding dimensions must match the database schema.

The `pgvector` column is declared as `vector(n)`. If the embedding model changes and produces a different vector dimension, the stored schema must match the new output shape. That may require recreating the database volume or running a migration.

This is a critical platform lesson. Embedding models are not interchangeable without operational consequences. The platform needs to track embedding model name, version, dimension, and re-indexing requirements.

Postgres and pgvector as the Vector Store

The project uses Postgres with `pgvector` as the vector database.

This is an intentional architecture choice.

Dedicated vector databases can be powerful, but Postgres with `pgvector` is an excellent path for learning and prototyping because it combines relational metadata and vector similarity search in one system.

That combination matters.

RAG systems do not only need vectors. They also need document metadata, source paths, timestamps, content hashes, chunk indexes, and operational fields.

Postgres gives the system a durable relational foundation.

`pgvector` adds semantic search capability.

Together, they allow the platform to store both the source record and the vector representation.

The schema has two primary tables.

`documents`

The `documents` table stores one row per source document.

It includes:

- document ID
- source path
- title
- content hash
- metadata
- created timestamp

`document_chunks`

The `document_chunks` table stores the searchable chunks.

It includes:

- chunk ID
- parent document ID
- chunk index
- chunk text
- token estimate
- embedding vector
- metadata
- created timestamp

This design preserves the relationship between the original document and the chunks used for retrieval.

That relationship is essential for citation, debugging, explainability, and source traceability.

A RAG assistant should not simply return an answer. It should be able to explain which source chunks supported that answer.

Vector Search

The vector search layer embeds the user's query, compares that query embedding against stored chunk embeddings, applies a similarity threshold, orders results by nearest vectors, and returns the top matches.

The search endpoint is:

`POST /search`

This endpoint is useful even without answer generation. It allows the developer to inspect whether retrieval is working before involving the LLM.

That separation is important.

If the assistant gives a weak answer, the problem may not be the model. The problem may be retrieval.

The top-ranked chunks may be irrelevant.

The threshold may be too high.

The threshold may be too low.

The chunk size may be wrong.

The embedding provider may not be strong enough.

The query may be vague.

The search endpoint gives the platform a way to debug retrieval directly.

The core retrieval pattern uses cosine distance:

```
SELECT
  c.id AS chunk_id,
  d.title,
  c.chunk_index,
  c.content,
  1 - (c.embedding <=> :query_embedding) AS similarity
FROM document_chunks c
JOIN documents d ON d.id = c.document_id
WHERE 1 - (c.embedding <=> :query_embedding) >= :similarity_threshold
ORDER BY c.embedding <=> :query_embedding
LIMIT :limit;
```

The similarity score gives the platform an interpretable signal. It helps determine whether a retrieved chunk is close enough to the question to be useful.

That score also becomes valuable for debugging, filtering, prompt construction, and retrieval evaluation.

The `/ask` Endpoint

The source-grounded answer endpoint is:

```
POST /ask
```


The `/ask` flow performs the full RAG sequence:

1. Embed the user question.
2. Search `pgvector` for relevant chunks.
3. Build a prompt using retrieved source context.
4. Optionally call the local LLM API.
5. Return the answer, sources, and optionally the generated prompt.

This endpoint turns the retrieval layer into an assistant.

The important design choice is that `/ask` can still return retrieved sources and a placeholder answer even when `LLM_API_URL` is not configured.

That makes the retrieval system independently testable.

This is good architecture. It avoids coupling every part of the project to the model-serving layer. The system can validate ingestion, chunking, embeddings, vector search, and source selection before depending on answer generation.

When `LLM_API_URL` is configured, the assistant can send the final prompt to the local LLM wrapper:

```
04-rag-document-assistant
-> retrieve context from Postgres + pgvector
-> build grounded prompt
-> call local LLM API /generate
```

This creates a clean boundary between retrieval and generation.

The RAG assistant owns documents, chunks, embeddings, retrieval, and prompt context.

The model-serving layer owns generation.

That separation keeps the system easier to test, debug, and extend.

Prompt Design

The project includes a source-grounded prompt template:

```
You are a source-grounded document assistant.
Answer only from the supplied context.
If the context does not contain the answer, say that the documents do not
```

provide enough information.
Always cite source titles and chunk numbers when possible.

This prompt is intentionally constrained.

The assistant is not being asked to answer from general knowledge. It is being asked to answer from retrieved context.

That matters because one of the main goals of RAG is grounding.

A useful document assistant should know when the retrieved context is insufficient. It should not invent an answer just to appear helpful.

The prompt also asks the model to cite source titles and chunk numbers when possible. That gives the user a way to trace the answer back to evidence.

The project also includes similarity scores in the prompt context. This is helpful during debugging because it shows how strongly each retrieved chunk matched the query.

From an AI Data Platform Architect's perspective, prompt design is part of system design. The prompt defines how retrieved data is transformed into an answer. It also defines how the assistant should behave when evidence is weak.

Retrieval Quality

RAG quality depends heavily on retrieval quality.

The final answer can only be as good as the context retrieved.

A retrieval layer is working well when the top chunks are:

- relevant to the question
- specific enough to support the answer
- diverse enough to avoid repeating the same passage
- grounded with source metadata
- ranked in a way that matches human judgment

The project identifies several useful retrieval metrics:

Metric	Meaning
Recall@k	Whether a relevant chunk appears in the top <code>k</code> results
Precision@k	How many of the top <code>k</code> chunks are useful

Metric	Meaning
MRR	How high the first relevant chunk appears
Similarity score spread	Whether the top result is clearly stronger than lower-ranked results
Citation accuracy	Whether the final answer cites the right sources

These metrics are important because RAG should be evaluated as a system, not just as a model response.

A polished answer is not enough.

The platform needs to know whether the answer was grounded in the right source material.

This requires evaluating retrieval, prompt construction, and citation behavior.

Common Failure Modes

The project also documents common RAG failure modes.

These are not edge cases. They are normal operating conditions for RAG systems.

Failure	Likely Cause	Fix
No results	Threshold too high or no documents indexed	Lower threshold or ingest documents
Repetitive results	Chunk overlap too high	Lower overlap or add diversity reranking
Wrong source cited	Retrieved context is weak	Improve embeddings or add query rewriting
Good chunk ranked low	Chunk too large or query too vague	Tune chunk size or rewrite query
Slow search	Large index without tuning	Tune indexes, add filters, benchmark

These failure modes reinforce an important point: RAG quality is an engineering problem.

When a RAG assistant fails, the fix is not always “use a better model.”

The fix may involve chunking strategy, embedding provider, similarity threshold, metadata filters, reranking, query rewriting, document freshness, or access control.

The model is only one part of the system.

Why This Architecture Matters

This project is local, but the architecture maps directly to production AI data platforms.

The same pattern can support:

- internal knowledge assistants
- technical documentation search
- policy and governance assistants
- data catalog question answering
- analytics documentation search
- runbook and incident response assistants
- customer support knowledge bases
- project documentation assistants

The value comes from combining data platform practices with AI application patterns.

Postgres stores durable document and chunk metadata.

`pgvector` provides semantic retrieval.

FastAPI exposes controlled service boundaries.

Prompt templates enforce grounding.

Source citations make answers traceable.

Retrieval evaluation gives the team a way to measure quality.

That is why RAG belongs in the AI data platform architecture conversation.

It is not just a chatbot feature.

It is a pipeline that turns enterprise knowledge into retrievable, auditable, source-grounded context.

Production Improvements

The local implementation creates the foundation, but a production version would add several important capabilities.

Semantic Embeddings by Default

The hash embedding provider is useful for local smoke testing, but production RAG systems need semantic embeddings by default.

Embedding model selection should be treated as a platform decision. The system should track embedding model name, version, dimension, and re-indexing history.

Metadata Filters

Production retrieval often needs filters.

Users may need to search by document type, business domain, owner, freshness, access policy, or source system.

Metadata filters improve relevance and enforce boundaries.

Document Versioning

Documents change over time.

A production RAG platform needs to know which version of a document was indexed, when it was indexed, and whether the index is stale.

Versioning and re-indexing workflows are essential for trust.

Reranking

Vector similarity search is often a first pass.

A reranker can improve final context selection by reordering retrieved chunks before prompt construction.

This is especially useful when the first-stage vector search returns many partially relevant results.

Evaluation Datasets

RAG systems need evaluation datasets.

A test set of representative questions, expected source chunks, and acceptable answers allows the team to measure retrieval quality over time.

This is how teams prevent regressions when changing chunking, embeddings, prompts, or thresholds.

Authorization Filters

Enterprise RAG systems need access control.

The retrieval layer should only return chunks the user is allowed to see.

This is not optional in environments with sensitive documentation, PHI, PII, financial records, or proprietary content.

Observability

A production platform should track query latency, search latency, hit rate, no-result rate, source usage, similarity distributions, answer quality, and citation behavior.

Without observability, retrieval quality becomes difficult to manage.

Batch Ingestion

Large document collections require batch ingestion workflows.

A production system needs to handle many documents, failures, retries, stale indexes, and re-indexing jobs.

This is where the RAG assistant can eventually connect to broader background processing and orchestration patterns.

Final Takeaway

The `04-rag-document-assistant` project demonstrates the core architecture of a source-grounded AI assistant.

It loads documents, normalizes content, creates overlapping chunks, generates embeddings, stores vectors in Postgres with `pgvector`, retrieves relevant context, builds grounded prompts, and optionally connects to a local LLM service for answer generation.

The most important lesson is that RAG is a data system before it is a model interaction.

The answer quality depends on the entire retrieval pipeline:

- document loading
- normalization
- chunking
- embeddings
- metadata
- vector search
- similarity thresholds
- prompt construction
- source citation
- retrieval evaluation

For an AI Data Platform Architect, this project makes the moving parts of RAG visible and understandable.

It shows how local documents can become searchable, retrievable, source-grounded context — and how that foundation can evolve into reliable enterprise AI infrastructure.